Sakala AI + Mitra · Technical Architecture, end to end

Everything as built (26 Oct 2026): two browser front ends, two FastAPI services, one AI layer (voice agent, intent brain, NL→SQL, understanding and routing, document reading, knowledge base, guardrails), the shared 8-stage grievance pipeline and the SQLite data layer, with every model the project uses and what it falls back to. Every model call is Google Gemini since 23 Oct 2026; Kannada is full duplex on the same live session.

ಸಕಾಲ · ಮಿತ್ರ
Government of Karnataka
Praja Seva Department
Sakala POC :8020 · Mitra :8030 · 26 Oct 2026

Measured on the running system

- **0.45 s** — **first word** on the live voice (Kannada ~0.5 s); function call 0.4 s
- **0.7 s** — a change reaches **every other screen**
- **97.6%** — same officer as iPGRS on **673 real cases** (95.7% statewide on 4,611)
- **97.5%** — department **from text alone** on 40 real complaints (service 72.5%, exact officer 77.5%, Gemini 3.6 Flash)
- **12–17 s** — **handwritten letter** → official form, all gold fields right
- **113 / 113** — question bank · 32/32 real voice · 18/18 Kannada live · 59/59 guardrails

LAYERED ARCHITECTURE

browser → API → AI layer → pipeline → database · both channels share layers 3–5

1 · Frontend & channels

vanilla JS single-page apps, no framework, canvas renderers

WhatsApp citizen — POC :8020
- iPhone-framed WhatsApp: text, voice note, petition photo, location, lists, chips
- Two-way: officer messages polled every 3.5 s; 👍 / 👎, reply, appeal, track inside the chat
- "Behind the chat": the 8 stages light up live; upload a written complaint (modal + form)

`app_wa_client.js` · `docupload.js`

Officer & CS console — POC :8020
- Officer inbox, case drawer, accept / ask / return / reject / resolve, ATR check, compensation
- CS dashboard: Karnataka map (KGIS districts + OSM taluks, SVG, zoom to dots), KPIs, Sakala tracker, Wednesday pack
- AI mode: ask the data; Chart.js 4.5.1 bar / line / donut

generated from `wireframe/index.html` by `make_app_ui.py`

Mitra web desk — Mitra :8030
- Three desks (Citizen / Officer / CS), one voice; particle orb, word-by-word captions, cards, notifications
- Profile portal, splash, light / dark, 8-stage rail, conversation strip with replayable audio
- Stage mode: canvas 3D bars, race, line, donut and KaMap, revealed as names are spoken

`mitra.js` · `orb.js` · `stage.js` · `portal.js` · `splash.js`

Mitra phone app — same server
- The real Mitra in an iPhone frame (393×852); bottom sheet with tabs that rises for what Mitra did
- Composer at the thumb, paperclip upload, camera; web ⇄ app by voice
- "Behind the app" panel mirrors the pipeline via `postMessage`

`app.js` · `phone.js`

Audio I/O — Web Audio
- Mic → 16 kHz PCM frames; playback 24 kHz PCM queue with instant cut
- "Hey Mitra" scanner in every state; echo guard, ducking, barge-in
- Mute / push-to-talk; 16 kHz WAV clips for the cascade

no browser speech service needed

↑↓ HTTP / JSON · polling

↑↓ WebSocket (audio + events) · SSE · HTTP

2 · API layer

Python 3.11 · FastAPI · uvicorn · httpx

Sakala POC service — :8020 · app/server/api.py
- WhatsApp engine `/api/chat` · state `/api/state`, `/api/case/{id}` · officer actions `/api/cases/{id}/action`, `draft_atr`
- Shared intake for every non-WhatsApp door `/api/intake`: understand · place · create · scrutiny · document · apply · reply · feedback · appeal
- Data `/api/query` (9 tools) · `/api/ask` (AI mode) · insights, tracker, early warning, review pack, E-4, upload, transcribe · change feed `/api/changes` · clock, reset

Mitra service — :8030 · mitra/server/main.py
- `/ws/live` full-duplex relay to the live voice model (key stays on the server)
- Cascade turn `/api/turn` (audio or text) · `/api/tts`, `tts_part` · `/api/document`, `/submit` · `/api/photo`
- Real time `/api/events` (SSE) · `/api/notifs` · voice notes · wake-word learning · `/api/forget` · drives the POC over HTTP (`sakala.py`)

Cross-cutting — middleware
- Per-IP rate limits (chat 120/min, ask 60, actions 120, upload 20)
- 8 MB request cap, security headers, CORS, no-cache on pages
- Debug surfaces only with `MITRA_DEBUG=1`

↑↓ every AI call logged (ai_log)

↑↓ one code path: tools.execute for Live and cascade

3 · AI layer

models choose from closed lists; facts and numbers come from data

Voice agent — live.py · speech.py
- Live: native-audio model, duplex, function calling, English and Kannada (kn-IN), per-desk persona + screen context
- Cascade: speech-to-text → brain → text-to-speech; typed input, the wake scanner and the fallback path
- Standby → Live → Standby (20 s)

Intent brain — brain.py · tools.py
- 55 intents (27 / 39 / 34 per desk); one per turn
- Rules first (navigation, upload, charts, theme, data, knowledge, verdicts), then model → guided JSON → rules
- Read-back before consequential actions

NL → SQL — query.py · compare.py
- Question → one of 10 typed tools (or AI mode: ≤3 of 5 tools)
- Fixed parameterised SQL; the model **never writes SQL**
- Officer scope applied in SQL; charts built from rows

Understanding & routing — ai.py · local_ai.py
- Masked text + 5 nearest real cases → Gemini 3.6 Flash → dept › service › grievance (enum from masters, JSON schema)
- Offline model must agree; keyword rules override; threshold tuned from accuracy
- Learned role rules → named officer

Document AI — docread.py
- Orientation: 4-turn sheet, 2 voters
- 3 parallel reads (transcript || fields + English || check); disagreements flagged
- Letter's own iPGRS no. retrieved as evidence; form = `ipgrs_form.json`

Knowledge & guardrails — knowledge.py · guard.py
- `knowledge.yaml` (34 notes), deterministic match; else "not in my notes"
- Injection stripped, vendor names scrubbed, identity fixed, spoken ids / numbers checked
- Masking before any AI step

↑↓ /api/intake · /api/query · /api/ask

↑↓ same functions as WhatsApp (chat.place_reply, create_grievance)

4 · Core services

the shared grievance pipeline + clocks + real time

8-stage pipeline (every channel) — pipeline.py · routing.py · chat.py

1 Entry: WhatsApp, Mitra voice / text, letter, photo, upload → 2 Privacy: phone, Aadhaar, email, card no. masked → 3 Understand: master-only, confidence, reason → 4 Complete: village / GP / taluk, GPS, checklist → 5 Validate: duplicate, wrong door, confirm → 6 Route: role + taluk + GP → named officer → 7 Clock: SLA, reminders, escalation → 8 Leaders: officer + CS screens, map

Unclear / outside the pilot / below the bar → scrutiny desk (EO of the taluk), filed only on a yes · every decision's evidence stored in `cases.audit`

Clock & accountability — sla.py
- L1 → L2 → L3 auto-escalation (7 / 5 / 3 days, assumed)
- Sakala legal clock, ₹20/day ≤ ₹500, Form C → HRMS (mock)
- Resolution loop: "not solved" → one rung up

Real time & analytics — sync · notify · analytics
- `sync.py` tails changes every 0.7 s → `notify.py` by role → SSE
- Hotspots, misrouting, data quality, accuracy, tracker, early warning, review pack

↑↓ auto-commit · WAL

↑↓ ingest.py builds everything from the real Excel

5 · Data layer

SQLite (WAL) · files · source data

sakala.db — app/data · written the moment a case is filed

cases 4,763 · case_events 6,394 · notifications · citizen_msgs · chat_sessions · chat_messages · ipgrs_records 9,999 (masked) · officers 3,433 · gazetteer 2,371 · departments 38 · services 124 · grievance_types 221 · sakala_services · sakala_perf 384 · ai_log · kv (clock, evals, thresholds)

`cases.audit` = routing evidence · `cases.doc` = the letter's form · reset from `sakala_pristine.db`

mitra.db — mitra/data

sessions · turns (7-day retention) · voice_notes · feed · seen cursors · deviations

"forget this conversation" deletes a session's turns

Files & models on disk

local_models.joblib · llm_eval.json · knowledge.yaml · ipgrs_form.json · wake_variants.json · mapdata.js (31 districts) · .env (keys, server only)

Source data → ingest.py — ~2 min

9,999 iPGRS grievances, 3–9 Oct 2026 · Sakala 2025 district reports · KGIS / OSM boundaries · full LGD master · live iPGRS / HRMS APIs

masters, learned officer rules, gazetteer, officers, caseload, offline model, routing eval

A · Every model in the project

MODEL	USED FOR	FALLS BACK TO
Gemini **gemini-3.8-live**	Mitra full-duplex voice + function calling, English and Kannada (kn-IN)	cascade (5 min)
Gemini **gemini-3.8-flash**	Mitra brain (cascade, guided JSON), Kannada rendering of replies, letter-reader fallback	3.5-flash → 3.5 / 3.1-lite → rules
Gemini **gemini-3.6-flash**	POC understanding (enum JSON), petition OCR, ATR check + draft, AI-mode planner + writer, Kannada for officers, briefs; letter reads ×3 in parallel; orientation voter	3.1-flash-lite → offline model
Gemini **gemini-3.1-flash-lite**	Mitra speech-to-text + wake check; WhatsApp voice notes, AI-mode mic; orientation voter	3.5-flash-lite → 3.8-flash / 3.6-flash
Gemini **gemini-3.8-flash-tts**	Mitra cascade voice only (Kore); replies over HTTP are read by the live voice instead	lite-tts → 3.1-tts → browser
Local **TF-IDF + SGD**	38 departments / 124 services, 5 nearest real cases (retrieval), keyword rules, Kannada transliteration; must agree before auto-routing	always on
Local **rules**	wrong door, out of scope, Sakala requests, intents (EN + KN), knowledge match, question router	always on
Removed **qwen4-deploy · qwen3-asr · qwen3-tts**	the office H200 gateway; removed 23 Oct 2026, nothing calls it	—
Idle **Sarvam saaras / bulbul**	adapters written; only by explicit setting, no key	not active

Gemini only. Mitra: a 429 parks a model for 30 min, a 503 for 2 min, then the next in the chain. POC: Gemini's OpenAI-compatible endpoint, JSON schema, reasoning effort low; 429 / 5xx / timeouts fall through the chain, no network backs off 60 s to the offline model. On screen and in speech the engine is only ever "Mitra" / "Sakala AI".

B · NL → SQL (ask the data)

Mitra, by voice
1. "Which district has the most overdue cases, on the map?"
2. Deterministic router (`route_question`) or the model picks a tool + typed args (district, taluk, dept, status, metric, group_by, chart)
3. POC `/api/query` runs fixed, parameterised SQL; officer scope applied; names resolved (Mysore → Mysuru)
4. Rows + per-district geo → answer + chart card; numbers spoken are checked against the rows

POC AI mode, typed
1. Question guard: overrides and off-topic refused, logged
2. Planner picks ≤3 of: query_cases, real_records, record, sakala_2025, insights
3. Writer drafts HTML only from tool results → sanitised; chart from the rows

The model never writes SQL and never invents a number.

SQL lives in `app/server/query.py` (lookup, find, iPGRS, officer, place, statewide, Sakala 2025), `compare.py` (compare, ranking with per-district geo) and `analytics.py` (insights, tracker, review pack).

C · Voice agent, one turn

1. "Hey Mitra" / tap / type → Live opens (setup 1.3 s)
2. 16 kHz audio streamed through `/ws/live` ; transcript both ways
3. Model calls a function (0.4 s) → `tools.execute` → POC
4. Result `{say, data, ui}` sanitised → spoken (first word 0.45 s), cards and charts drawn
5. Barge-in cuts it at once; HTTP replies (letter read, notification) read in the same voice
6. 20 s silence → Standby; failure → cascade, same functions; a language switch reopens Live in kn-IN / en

Cascade

16 kHz WAV → speech-to-text → rules / model brain → function → text-to-speech, sentence by sentence (first part arrives with the turn).

Real-time loop across desks
1. Officer resolves by voice → POC writes `cases` + `case_events`
2. `sync.py` sees it within 0.7 s → `notify.py` picks who is told
3. SSE → the citizen's screen: card + Mitra says it in the live voice; "not solved" → one rung up

D · Guardrails & verification

- Only official masters; else a person decides
- Masking before any AI step
- Acts only through functions; read-back first
- Officers see their ladder; citizens their own
- Ladders never mix; compensation only Sakala
- Junk never filed; unclear → scrutiny on a yes
- Every action on the record "via Mitra (voice)"
- 7-day retention, "forget this conversation"

Test suites (green)

bank 113 · battery 101 · guardrails 59 + 27 · real Live 32 · Kannada live 18 · letters 28 · return 31 · statewide 40 · parity 14 · knowledge 14

Headless Edge over the DevTools protocol for every UI; fake Live server for offline runs.

Run & deploy

launchers (Brave tab) · run.bat → :8020 / :8030 · test instances :8031 / :8032 · keys only in .env · AWS: systemd services + Caddy HTTPS gate

Two front ends, one pipeline • Models choose, data decides • Every number from SQL • Every action on the record • Live on every screen